

Folio

Seu sistema de produtividade. Do seu jeito.

Documento de Produto • Versão 1.0

1. Visão geral

Folio é uma plataforma de produtividade pessoal construída sobre uma filosofia simples: o app não impõe nenhum método. Ele entrega as ferramentas fundamentais que qualquer pessoa precisa para se organizar — e deixa o usuário livre para montar o sistema que funciona pra ele.

Em vez de obrigar o usuário a usar Pomodoro, Bullet Journal ou GTD, o Folio oferece um core sólido de quatro módulos nativos e uma loja de plugins open source onde a comunidade — e o próprio usuário — pode estender a plataforma com os métodos que preferir.

A metáfora é o VSCode da produtividade: poderoso por padrão, infinito por extensão.

2. O problema que o Folio resolve

Pessoas que levam produtividade a sério hoje usam em média 3 a 5 aplicativos diferentes — um para notas, outro para tarefas, outro para hábitos, outro para agenda. O problema não é falta de opções. É excesso de fricção entre elas.

As alternativas existentes falham de formas diferentes:

- Notion é poderoso demais e genérico demais — exige que o usuário construa tudo do zero, o que afasta quem não tem tempo ou disposição pra isso.
- Obsidian é excelente para PKM técnico, mas tem curva de aprendizado enorme e é focado em texto, não em sistema.
- Todoist, TickTick e similares são ótimos para tarefas, mas não integram notas, hábitos ou agenda de forma coesa.
- Apps de Bullet Journal digital existem, mas nenhum integra hábitos, revisões ou agenda no mesmo lugar.

O Folio resolve isso entregando uma base funcional e coesa — e permitindo que o usuário expanda com o que faz sentido pra ele, sem precisar construir nada do zero.

3. Filosofia do produto

Três princípios guiam cada decisão de produto no Folio:

- **Opinionado no core, livre na extensão.**
 - Os módulos nativos têm estrutura clara e funcional. O usuário não configura nada pra começar. Mas tudo que vai além disso é escolha dele.
 - **Integração real, não apenas coexistência.**
 - Plugins não são ilhas isoladas. Eles leem e escrevem nos dados do core — uma tarefa criada no plugin GTD aparece no Daily Log; um hábito marcado aparece no timeline da agenda.
 - **A plataforma cresce com a comunidade.**
 - Qualquer desenvolvedor pode criar e publicar um plugin. O Folio modera a loja, mantém o core e define o contrato de API — a comunidade constrói o ecossistema.
-

4. Módulos nativos

Estes quatro módulos estão disponíveis para todos os usuários assim que criam uma conta. Não requerem instalação, configuração ou escolha de método.

4.1 Daily Log

O Daily Log é o ponto de entrada diário do Folio. Inspirado na estrutura do Bullet Journal, mas sem obrigar nenhuma notação específica, ele é um registro livre do dia — tarefas, notas rápidas, eventos e pensamentos, todos no mesmo lugar.

Como funciona

- Ao abrir o Folio, o usuário cai diretamente no Daily Log do dia atual.
- Cada entrada pode ser marcada como tarefa, nota ou evento — mas essa marcação é opcional.
- Tarefas não concluídas ao final do dia são destacadas para migração manual no dia seguinte. O usuário decide o que migra, o que descarta e o que agenda para uma data futura. Não existe migração automática.
- O histórico de dias anteriores é acessível por um calendário lateral — o usuário pode navegar e editar qualquer dia.
- Entradas do Daily Log alimentam o timeline da Agenda Semanal automaticamente.

Integração com plugins

Plugins que criam tarefas ou registros (GTD, Pomodoro, Bullet Journal avançado) podem injetar entradas no Daily Log via API interna. O usuário vê tudo num único lugar, independente de onde foi criado.

4.2 Notas e revisões

O módulo de Notas é um editor leve, sem blocos de construção complexos, sem banco de dados, sem relações entre páginas. Ele foi desenhado para capturar pensamentos, resumos, aprendizados e referências com o mínimo de fricção.

Como funciona

- Notas são organizadas por pastas livres — o usuário cria a estrutura que quiser.
- Cada nota tem um editor de texto com suporte a Markdown básico: negrito, itálico, títulos, listas e blocos de código.
- Notas podem ser vinculadas a entradas do Daily Log — ao escrever sobre uma reunião, o usuário pode referenciar a nota com os pontos discutidos.
- O módulo de revisões é uma camada sobre as notas: o usuário pode marcar qualquer nota para revisão futura e definir uma data. Na data marcada, a nota aparece no Daily Log com um lembrete.
- Revisões não implementam espaçamento automático por padrão — essa lógica fica para um plugin da comunidade. O core oferece apenas o agendamento manual.

Integração com plugins

O plugin de Revisão Espaçada, por exemplo, pode ler as notas marcadas para revisão e calcular automaticamente as datas seguintes com base no desempenho do usuário — sem que o usuário precise sair do módulo de notas nativo.

4.3 Hábitos e metas

O módulo de Hábitos é uma ferramenta de acompanhamento de comportamentos recorrentes. Metas são objetivos maiores, de médio e longo prazo, que agrupam hábitos e tarefas relacionados.

Hábitos — como funciona

- O usuário cria um hábito com nome, frequência (diário, dias da semana específicos, semanal) e uma cor.
- A marcação de conclusão é feita com um clique — do Daily Log, da tela de hábitos ou do widget do dashboard.
- Cada hábito exibe um streak — sequência de dias consecutivos concluídos — e um histórico de 30 dias em formato de grade, similar ao contribution graph do GitHub.
- Hábitos marcados no dia aparecem automaticamente no timeline da Agenda Semanal.

Metas — como funciona

- Uma meta tem título, prazo e uma lista de critérios de conclusão — que podem ser tarefas, hábitos ou checkpoints livres.
- O progresso da meta é calculado automaticamente com base na conclusão dos critérios vinculados.
- Metas aparecem no dashboard como cartões de progresso.

Integração com plugins

Plugins podem criar hábitos programaticamente — um plugin de rotina matinal, por exemplo, pode gerar automaticamente os hábitos da rotina quando instalado. Plugins também podem ler o histórico de hábitos para gerar análises e relatórios.

4.4 Agenda semanal e timeline

A Agenda Semanal é a visão consolidada da semana — não um calendário de compromissos, mas um painel que mostra tudo que aconteceu e vai acontecer, reunido num único lugar.

Visão semanal

- A semana é exibida em colunas de dias, cada uma com os eventos agendados, tarefas do Daily Log e hábitos marcados naquele dia.
- O usuário pode arrastar eventos e tarefas entre dias diretamente na visão semanal.
- Blocos de tempo podem ser criados manualmente — o usuário reserva períodos do dia para atividades específicas, sem que isso crie uma tarefa ou evento.

Timeline de atividade

Esta é a funcionalidade visual mais distintiva do Folio. Inspirada no contribution graph do GitHub, a timeline exibe um calendário anual em que cada dia tem uma intensidade de cor proporcional à quantidade de atividade registrada naquele dia.

- Atividade é calculada a partir de: entradas no Daily Log, tarefas concluídas, hábitos marcados e registros de plugins ativos.
- Ao passar o mouse sobre um dia, o usuário vê um resumo rápido: X tarefas concluídas, X hábitos marcados, X notas criadas.
- A timeline é visível na Agenda Semanal e também no perfil do usuário, como um retrato visual da consistência ao longo do tempo.

- Plugins podem contribuir com eventos para a timeline via API — um plugin de leitura, por exemplo, pode registrar os dias em que o usuário leu, e esses dias aparecem coloridos na grade.

Integração com plugins

A timeline é o ponto de convergência de todo o ecossistema. Tudo que acontece no Folio — core ou plugin — pode deixar uma marca na timeline. Isso cria um retrato honesto e completo da produtividade do usuário, independente do método que ele usa.

5. Loja de plugins open source

A loja de plugins é o coração extensível do Folio. Qualquer desenvolvedor pode criar e publicar um plugin — o Folio define o contrato, a comunidade constrói o ecossistema.

5.1 Filosofia da loja

- Todos os plugins são open source. O código é público, auditável e forkável.
- O Folio modera a loja: plugins passam por revisão antes de serem publicados para garantir que seguem o contrato de API e não comprometem dados do usuário.
- Plugins oficiais são mantidos pela equipe do Folio. Plugins da comunidade são mantidos pelos seus autores.
- O usuário instala, desinstala e configura plugins individualmente — cada um aparece como um módulo adicional no menu lateral.

5.2 Contrato de API interna

Todo plugin tem acesso à API interna do Folio, que expõe as seguintes capacidades:

Recurso	O que o plugin pode fazer	Exemplo de uso
Daily Log	Criar, ler e listar entradas do dia	Plugin GTD injeta próximas ações no log
Notas	Criar, ler, atualizar notas e tags	Plugin de revisão espaçada agenda notas
Hábitos	Criar hábitos e registrar conclusões	Plugin de rotina cria hábitos automaticamente
Tarefas	Criar, atualizar e completar tarefas	Plugin Pomodoro marca tarefa ao fim do ciclo
Timeline	Registrar eventos de atividade	Plugin de leitura marca dias de leitura
Dashboard	Adicionar widgets ao painel principal	Plugin de hábitos exibe streak no dashboard

5.3 Plugins iniciais previstos

Estes são os primeiros plugins que serão desenvolvidos — alguns pela equipe do Folio, outros abertos para contribuição da comunidade desde o início:

- Hábitos avançados (oficial) — streak detalhado, análise de padrões e notificações configuráveis.
- Revisão espaçada (oficial) — agenda automática de revisão de notas com base no algoritmo SM-2.
- Pomodoro (comunidade) — timer de 25/5 vinculado às tarefas do core.
- Bullet Journal completo (comunidade) — notação BuJo completa com collections, future log e migração avançada.
- GTD (comunidade) — implementação do Getting Things Done sobre o core de tarefas.
- Diário (comunidade) — registro de reflexões diárias com prompts e histórico de humor.
- Leitura (comunidade) — log de livros lidos com integração na timeline.

6. Stack técnica

A stack foi escolhida com dois critérios: alinhamento com o que você já está estudando (TypeScript, Node, Next.js) e capacidade de crescer para um produto real sem reescritas dolorosas.

6.1 Visão geral

Camada	Tecnologia e justificativa
Frontend	Next.js 14 + TypeScript — App Router, SSR para SEO, deploy automático na Vercel
Estilização	Tailwind CSS — produtividade de desenvolvimento e consistência visual
Componentes	shadcn/ui — componentes acessíveis, sem opinião visual, fáceis de customizar
Backend	Node.js + Express + TypeScript — API REST separada do frontend
Banco de dados	PostgreSQL via Supabase — gerenciado, plano grátis generoso, sem configurar servidor
ORM	Prisma — type-safe, migrations automáticas, integração nativa com TypeScript
Autenticação	JWT com refresh token — controle total, sem dependência de biblioteca externa
Tempo real	Supabase Realtime — para sincronização de dados entre abas e dispositivos
Email	Resend — moderno, templates em React, simples de integrar
Frontend host	Vercel — deploy automático a cada push no GitHub

Backend host	Railway — simples, barato, sem gerenciar servidor
CI/CD	GitHub Actions — build e testes automáticos

6.2 Arquitetura dos plugins

Esta é a parte mais delicada da stack — como plugins externos se integram ao core de forma segura e previsível.

Abordagem: plugins como módulos internos (MVP)

No MVP, plugins não são carregados dinamicamente em runtime. Eles são repositórios separados que seguem um contrato TypeScript definido pelo Folio. O processo de publicação envolve:

- O desenvolvedor cria o plugin seguindo o SDK open source do Folio.
- O plugin é submetido via Pull Request para o repositório oficial da loja.
- A equipe do Folio revisa, aprova e incorpora o plugin ao bundle da próxima versão.
- O usuário instala o plugin pelo painel — na prática, ativa um módulo que já está no bundle.

Essa abordagem é mais simples de implementar, mais segura e mais rápida de lançar. A complexidade de carregar código arbitrário em runtime (microfrontends, sandboxing) fica para uma versão futura.

Contrato do plugin — interface TypeScript

```
interface FolioPlugin {
  id: string
  name: string
  version: string
  author: string
  description: string
  onInstall?: (api: FolioAPI) => Promise<void>
  onDailyLogOpen?: (api: FolioAPI, date: string) => Promise<void>
  dashboardWidget?: React.ComponentType<{ api: FolioAPI }>
  sidebarItem?: { label: string; icon: string; page: React.ComponentType } }

```

FolioAPI — o que o plugin acessa

- `api.dailyLog.addEntry(date, content, type)` — adiciona entrada no log
- `api.notes.create(title, content, tags)` — cria uma nota
- `api.habits.create(name, frequency)` — cria um hábito
- `api.tasks.create(title, dueDate)` — cria uma tarefa
- `api.timeline.addEvent(date, label, color)` — marca um evento na timeline
- `api.dashboard.addWidget(component)` — adiciona widget ao painel

7. Estrutura de pastas

Frontend — folio-web/

```
folio-web/ |— app/ | |— (auth)/ | |— login, cadastro | |— (app)/ | | |
|— daily-log/ |— módulo daily log | | |— notes/ |— módulo notas | | |
|— habits/ |— módulo hábitos | | |— agenda/ |— agenda + timeline | | |
| |— plugins/ |— loja e gerenciamento | |— components/ | |— ui/
| |— shadcn/ui base | |— core/ |— componentes dos módulos | |—
plugins/ |— widgets de plugins ativos | |— lib/ | |— api.ts |—
chamadas ao backend | |— plugin-api.ts |— FolioAPI para plugins | |—
auth.ts |— plugins/ |— código dos plugins aprovados

```

Backend — folio-api/

```

folio-api/ ├── prisma/ | └── schema.prisma ← definição das tabelas | └── src/ |
| ├── controllers/ ← recebe req HTTP | └── services/ ← regras de negócio
| ├── repositories/ ← acesso ao banco | └── middlewares/ ← auth JWT,
validação | └── routes/ ← definição das rotas | └── config/ └── docs/
| └── decisions/ ← ADRs | └── business-rules.md └── README.md

```

8. Roadmap de desenvolvimento

Fase	Prazo estimado	Entregável
MVP	Semanas 1–8	Auth, Daily Log, Notas, Hábitos básicos, Agenda semanal, Timeline
V2	Semanas 9–14	SDK de plugins, primeiros 2 plugins oficiais, loja básica, metas
V3	Semanas 15–22	Plugins da comunidade, monetização, mobile PWA, sync multi-dispositivo

9. Monetização — hipóteses iniciais

A monetização ainda está em aberto — e isso é saudável. Antes de definir um modelo, o Folio precisa entender como os usuários usam o produto e o que eles valorizam. Dito isso, estas são as hipóteses mais promissoras:

- **Freemium com limite de plugins ativos.**
 - Plano free: core completo + até 2 plugins ativos. Plano pro: plugins ilimitados + sync entre dispositivos + acesso antecipado a novos plugins.
- **Freemium com limite de dados históricos.**
 - Plano free: acesso aos últimos 90 dias de dados. Plano pro: histórico completo, exportação e backup.
- **Modelo de contribuição para plugins premium.**
 - Desenvolvedores podem publicar plugins pagos na loja — o Folio fica com uma comissão (tipo App Store). Open source não significa gratuito.

A recomendação é lançar o MVP gratuito, coletar dados de uso e validar o que os usuários estão dispostos a pagar antes de implementar qualquer paywall.

10. Próximos passos

Com o produto definido em documento, o caminho natural é:

- Criar o repositório e configurar o monorepo (folio-web + folio-api).
- Fazer o system design completo — modelagem do banco, contratos de API, fluxos de dados entre módulos e plugins.

- Desenvolver o MVP nas férias seguindo o roadmap da seção 8.
- Lançar uma landing page com lista de espera antes de terminar o MVP — validar interesse antes de publicar.
- Definir o modelo de monetização com base nos primeiros 100 usuários reais.

Folio — construído com método, sem impor nenhum.